

Controle de Tempo Avançado e Interrupções

O Problema das Funções Bloqueantes (delay e delayMicroseconds)

A função `delay(ms)` suspende a execução do programa pelo tempo especificado em milissegundos. Durante esse tempo, o microcontrolador fica "cego" e "surdo": ele não lê sensores, não processa comandos e não responde a botões.

- `delay(1000);` // Trava o processador por 1 segundo (1000 milissegundos).
- `delayMicroseconds(50);` // Trava o processador por 50 microssegundos ($1\text{ us} = 0.000001\text{s}$).
É usada quando precisamos de pulsos extremamente rápidos (como o gatilho do sensor Ultrassônico).

Programação Não-Bloqueante: `millis()` e `micros()`

Para resolver o problema do travamento, usamos o conceito de cronômetro. As funções `millis()` e `micros()` não travam o código; elas apenas informam há quanto tempo o microcontrolador está ligado.

- `millis()`: Retorna o número de milissegundos desde que o programa começou.
- `micros()`: Retorna o número de microssegundos desde o início.

O Tipo de Dado `unsigned long`

Como o tempo passa muito rápido, o número retornado por essas funções cresce muito. Por isso, nunca use `int` para armazenar o tempo. Use sempre `unsigned long` (inteiro longo sem sinal), que consegue armazenar valores por até 50 dias antes de zerar (overflow).

A Lógica do Cronômetro (Piscar sem travar)

Em vez de dizer "espere 1 segundo", o código diz: "A hora atual menos a última vez que o LED piscou é maior ou igual a 1 segundo?"

```
unsigned long tempoAnterior = 0;
const long intervalo = 1000; // Tempo de espera (1 segundo)
bool estadoLED = LOW;

void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  unsigned long tempoAtual = millis(); // Tira uma foto do relógio atual

  if (tempoAtual - tempoAnterior >= intervalo) {
    tempoAnterior = tempoAtual; // Salva o último momento em que o LED mudou

    estadoLED = !estadoLED; // Inverte o estado do LED
    digitalWrite(13, estadoLED);
  }

  // Qualquer outro código colocado aqui rodará IMEDIATAMENTE, sem travar!
}
```

Timers e Contadores Internos (Hardware)

Por trás das funções de software, existem periféricos de hardware dentro do chip chamados Timers/Counters. No Arduino Uno (ATmega328P), existem três:

- Timer0 (8 bits): Usado internamente para as funções millis(), micros() e para o PWM nos pinos 5 e 6.
- Timer1 (16 bits): Usado pela biblioteca de Servomotores (Servo.h) e para gerar frequências de som precisas. PWM nos pinos 9 e 10.
- Timer2 (8 bits): Usado pela função tone() (sons com buzzer). PWM nos pinos 3 e 11.

Eles contam os pulsos do cristal de clock do microcontrolador (geralmente 16 MHz). Quando a contagem estoura (overflow), eles geram um sinal.

Interrupções Externas (Hardware Interrupts)

Uma Interrupção é um sinal de hardware que força o microcontrolador a parar IMEDIATAMENTE o que está fazendo no `loop()`, executar uma função especial de emergência, e depois voltar exatamente para onde tinha parado.

Pense nisso como um alarme de incêndio: não importa o que você está fazendo, você para e atende ao alarme.

Pinos de Interrupção no Arduino Uno

Não são todos os pinos que aceitam interrupção por hardware. No Arduino Uno, temos apenas dois:

- Pino Digital 2 (Interrupção 0)
- Pino Digital 3 (Interrupção 1)

Modos de Disparo

Podemos configurar a interrupção para disparar em 4 momentos diferentes do sinal do pino:

- LOW: Dispara sempre que o pino estiver em nível baixo.
- CHANGE: Dispara sempre que o sinal mudar (de 0 para 1, ou de 1 para 0).
- RISING: Dispara na borda de subida (quando o botão passa de LOW para HIGH).
- FALLING: Dispara na borda de descida (quando o botão passa de HIGH para LOW).

Exemplo Prático de Interrupção

Para usar uma interrupção, usamos a função `attachInterrupt()` dentro do `setup()`:

```
const byte pinoBotao = 2;
volatile bool botaoPressionado = false; // 'volatile' é obrigatório para variáveis
usadas em interrupções

void setup() {
  pinMode(pinoBotao, INPUT_PULLUP); // Usa o resistor interno do Arduino

  // attachInterrupt(pino_da_interrupcao, funcao_a_ser_chamada, modo)
  attachInterrupt(digitalPinToInterrupt(pinoBotao), servicoEmergencia, FALLING);
}

void loop() {
  // O loop pode estar cheio de delays ou tarefas pesadas...
  delay(5000);
}
```

```
// Esta é a ISR (Interrupt Service Routine)
void servicoEmergencia() {
    botaoPressionado = true;
    // ATENÇÃO: Dentro de uma ISR, a função delay() NÃO funciona e millis() não
    avança!
    // Mantenha esta função o mais curta e rápida possível.
}
```

Exercícios de Fixação

Entregar individualmente.

Exercício 1: Lógica do Millis.

Um sistema de irrigação automatizado precisa ler a umidade do solo a cada 5 segundos (5000 ms) e piscar um LED de status a cada 200 milissegundos. Explique por que é impossível implementar esse sistema de forma confiável utilizando apenas a função delay().

Exercício 2: Análise de Código.

O código abaixo foi criado por um aluno para tentar desligar um motor em caso de emergência usando uma interrupção no pino 2. Identifique os dois erros graves cometidos pelo aluno dentro da função desligarMotor().

```
void setup() {
    pinMode(2, INPUT);
    attachInterrupt(digitalPinToInterrupt(2), desligarMotor, FALLING);
}
void loop() { /* Código do robô */ }

void desligarMotor() {
    digitalWrite(motor, LOW);
    Serial.println("EMERGENCIA! Motor desligado.");
    delay(2000);
}
```

Exercício 3: Dimensionamento de Variáveis.

Sabendo que a função millis() retorna um tipo de dado unsigned long (32 bits), o valor máximo que ela consegue armazenar antes de voltar para zero (estouro/overflow) é de

$$2^{32} - 1$$

milissegundos. Faça o cálculo e demonstre aproximadamente quantos dias o Arduino pode rodar continuamente antes que esse contador seja zerado.

Respostas:

Se usarmos `delay(5000)` para a leitura do solo, o LED de status ficará travado e não conseguirá piscar no intervalo de 200ms. Se usarmos `delay(200)` para o LED, teremos dificuldade de sincronizar a leitura exata de 5 segundos sem acumular atrasos. O uso de `millis()` é obrigatório para gerenciar os dois tempos independentes.

Erros na ISR: 1) Tentou usar a função `delay()`, que não funciona e trava o microcontrolador dentro de uma interrupção. 2) Tentou usar comunicações lentas como `Serial.println()`. Funções de interrupção devem ser instantâneas, apenas mudando o estado de variáveis.

$$2^{32} - 1 = 4.294.967.295$$

milissegundos. Dividindo por 1000 = 4.294.967 segundos. Dividindo por 3600 (segundos em 1 hora) = 1193 horas. Dividindo por 24 (horas em 1 dia) = 49.7 dias.